Nalin Nagar

Milestone 3 Report

https://drive.google.com/drive/folders/1Eza3z93P__0pO-dkBXZG26VLHR3Euts8?usp=drive_link

0. **Baseline (Kernel Fusion Unrolling):**

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	*0.830134 ms*	*0.462796 ms*	*1.29293 ms*	*0.86*
1000	*5.61164 ms*	*3.36355 ms*	*8.97519 ms*	*0.886*
10000	*54.855 ms*	*33.1395 ms*	*87.9945 ms*	*0.8714*

1. **Req_0: Using Streams to overlap computation with data transfer**
   https://drive.google.com/drive/folders/10mnzgKF5Av_IRGkorVbtglwuPQTeQACI?usp=sharing
   a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

      Streams optimize the unfused convolution pipeline by splitting the batch into chunks and assigning each chunk to a separate CUDA stream. While one stream is running the unroll, matrix multiply, or permutation kernels, another stream can asynchronously copy input/output data between host and device. This hides some data-transfer cost behind computation that was originally the bottleneck with the unfused convolution, (hopefully)reducing overall op time even though the convolution kernel itself is unchanged.

   b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.

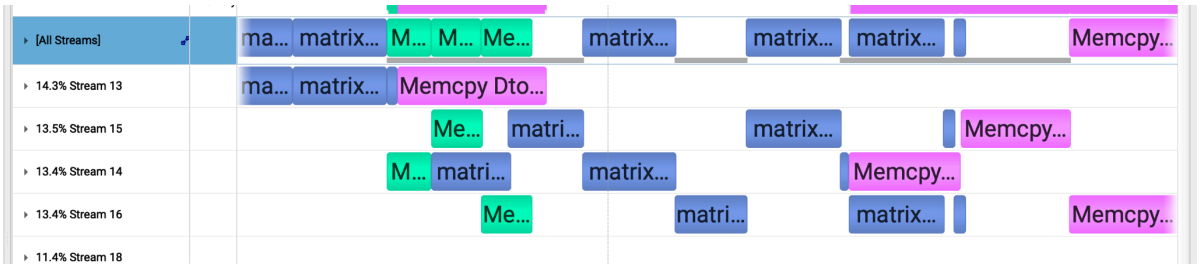
      Stream Creation:

```
const int stream_count = std::max(1, std::min(STREAM_COUNT,
Batch));
const int chunk_batch = (Batch + stream_count - 1) / stream_count;

std::vector<cudaStream_t> streams(stream_count);
```

Async Memory Copy:

```
cudaMemcpyAsync(device_input_chunk, host_input_chunk, input_bytes,
                cudaMemcpyHostToDevice, streams[s]);

matrix_unrolling_kernel<<<unroll_grid_dim, unroll_block_dim, 0,
streams[s]>>>(
    device_input_chunk, unrolled_matrix[s], curr_batch, Channel,
Height, Width, K
);

matrixMultiplyShared<<<matmul_grid_dim, matmul_block_dim, 0,
streams[s]>>>(
    device_mask, unrolled_matrix[s], matmul_output[s], Map_out,
Height_unrolled,
    Height_unrolled, (int)width_unrolled, Map_out,
(int)width_unrolled
);

matrix_permute_kernel<<<permute_kernel_grid_dim,
PERMUTE_BLOCK_SIZE, 0, streams[s]>>>(
    matmul_output[s], device_output_chunk, Map_out, curr_batch,
out_image_size
);

cudaMemcpyAsync(host_output_chunk, device_output_chunk,
output_bytes,
                cudaMemcpyDeviceToHost, streams[s]);
```

Timeline from Nsys:



The Nsight Systems timeline confirms that the streaming implementation overlaps data transfer with computation. Green blocks represent host-to-device transfers, blue blocks represent kernel execution, and pink blocks represent device-to-host transfers. Across Streams 13–16, transfers in one stream occur at the same time as kernel execution in other streams, showing that the batch chunks are pipelined rather than executed fully serially. This validates the streams optimization because the H2D/D2H transfer cost is partially hidden behind the unroll, matrix multiplication, and permutation kernels.

c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	*10.5579 ms*	*3.97499 ms*	*14.53289 ms*	*0.86*
1000	*19.59 ms*	*17.8995 ms*	*37.4895 ms*	*0.886*
10000	*103.155 ms*	*92.2982 ms*	*195.4532 ms*	*0.8714*

For streams, the performance did not match the ideal expectation. In theory, splitting the batch across streams should overlap host-to-device copies, kernel execution, and device-to-host copies, reducing total layer time. However, the measured times are much worse than the baseline, especially at batch size 10,000, where total execution time is 195.4532 ms.

The Nsight Systems timeline shows that transfers and kernels do overlap across streams, so the streaming mechanism is working. But each stream still launches the full unfused pipeline: input unrolling, matrix multiplication, permutation, and output copy. That adds extra kernel launches, extra global memory traffic from materializing the unrolled matrix, and extra temporary allocations for unrolled_matrix and matmul_output.

d. Does this optimization synergize with any other optimizations? How?

Streams synergize best with the unfused input-unrolling pipeline, not the fused kernel. They overlap host-to-device and device-to-host transfers with the unroll, matrix multiply, and permutation kernels from other batch chunks. So consequently streams do not strongly synergize with Tensor Cores, loop unrolling, or constant memory at the kernel level because streams improve transfer/compute overlap, not the arithmetic inside a kernel.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)

https://developer.nvidia.com/blog/how-overlap-data-transfers-cuda-cc/

https://developer.download.nvidia.com/CUDA/training/StreamsAndConcurrencyWebinar.pdf

2. **Req_1: Using Tensor Cores to speed up matrix multiplication**
https://drive.google.com/drive/folders/1Ma3OdUJvg6S6T3q0a19uY_PQQM1SfobT?usp=sharing

a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

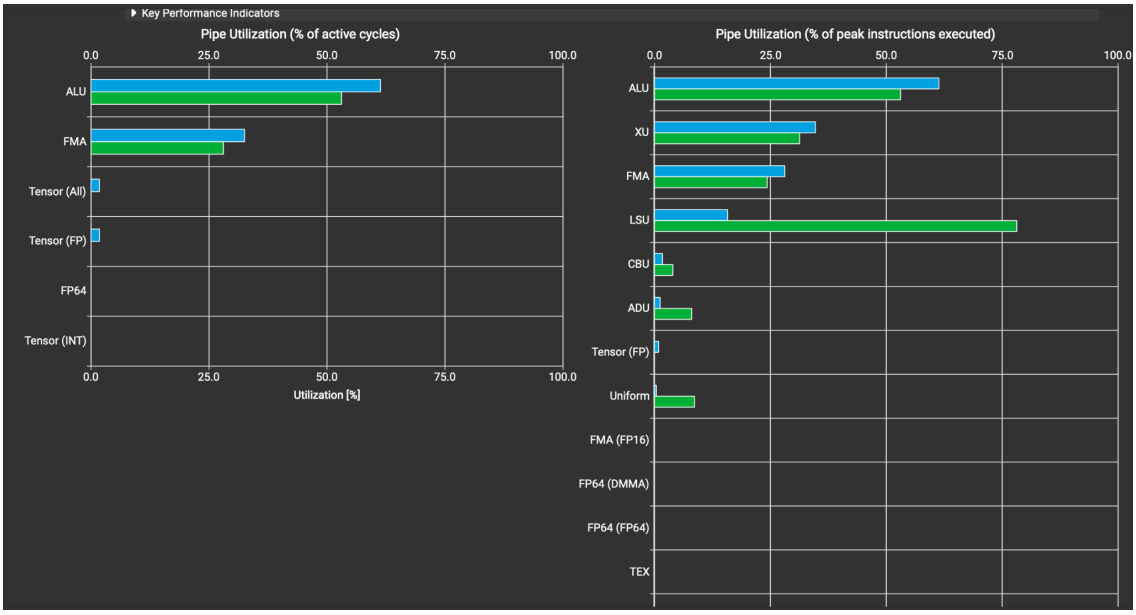
Tensor Cores optimize the convolution by accelerating the matrix multiplication inside the implicit-GEMM formulation. The kernel converts matrix tiles to FP16, loads them into WMMA fragments, and uses `wmma::mma_sync` to compute `16×16×16` tile products on Tensor Cores instead of regular FP32 scalar operations. In practice, both Conv1 and Conv2 can improve if the Tensor Core speedup outweighs the overhead from FP16 conversion, implicit indexing, and padded tile rows.

b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.

WMMA Tensor Core Instructions Snippet:

```
wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half,
wmma::row_major> a_frag;
wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half,
wmma::row_major> b_frag;
wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float>
c_frag;

wmma::load_matrix_sync(a_frag, tile_A, WMMA_K);
wmma::load_matrix_sync(b_frag, tile_B, WMMA_N);
wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);
```

NCU Profile Pipe Utilization(compared to baseline):



In the NCU Pipe Utilization section, the baseline implicit-GEMM kernel shows no visible Tensor pipe utilization, while the Tensor Core implementation shows nonzero Tensor pipe activity. This confirms that the WMMA implementation is exercising Tensor Core hardware. However, Tensor utilization remains small because the kernel still performs significant non-Tensor work, including implicit input indexing, FP32-to-FP16 conversion, shared-memory tile loading, and output scattering. Therefore, the optimization correctly uses Tensor Cores, but the speedup is limited by overhead outside the Tensor Core matrix multiply itself.

c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	*1.00421 ms*	*0.36839 ms*	*1.3726 ms*	*0.86*
1000	*5.16894 ms*	*3.40384 ms*	*8.57278 ms*	*0.886*

10000	*50.3944 ms*	*33.3792 ms*	*83.7736 ms*	*0.8714*

Yes, but only partially. I expected Tensor Cores to improve performance because the convolution is implemented as implicit GEMM, so the matrix multiply portion can be accelerated with WMMA. The results match that expectation for larger batches: at batch size 10,000, total execution time dropped from 87.9945 ms to 83.7736 ms, and Conv1 improved from 54.855 ms to 50.3944 ms. The improvement is there, but not anything crazy.

The profiling explains why. In NCU, the baseline showed essentially no Tensor pipe utilization, while the Tensor Core version showed a small nonzero Tensor pipe jump, meaning the WMMA path is actually using Tensor Core hardware. But the Tensor pipe utilization was still very low, while ALU utilization stayed high around 61%, so I believe the kernel is still spending a lot of time on non-Tensor work like implicit input indexing, FP32-to-FP16 conversion, shared-memory tile loading, and thread synchronization. That means Tensor Cores helped the matrix multiply portion a little, but the overall kernel is still bottlenecked by overhead around the GEMM computation.

For smaller batches, performance did not consistently improve. At batch size 100, total time increased from 1.29293 ms to 1.3726 ms, likely because the overhead of setting up WMMA tiles and converting values to FP16 outweighed the Tensor Core benefit. At batch size 1000 and 10000, the larger workload gave Tensor Cores more useful work, so the optimized version became faster.

d. Does this optimization synergize with any other optimizations? How?

   Tensor cores mainly synergize with FP16 because WMMA uses FP16 input fragments with FP32 accumulation. However, Tensor Cores may conflict with small tile shapes or Conv1-style dimensions because WMMA operates on fixed

16×16×16 tiles, so poor tile utilization can limit the benefit. In my experience it improved op-times to use both however the total op-times were still larger than the baseline, likely due to an inefficient implementation.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)

https://developer.nvidia.com/blog/programming-tensor-cores-cuda-9/

https://docs.nvidia.com/cuda/parallel-thread-execution/index.html?highlight=wmma#warp-level-matrix-instructions-wmma-ld

https://alexarmbr.github.io/2024/08/10/How-To-Write-A-Fast-Matrix-Multiplication-From-Scratch-With-Tensor-Cores.html

3. **Op_0: Weight matrix (Kernel) in constant memory**
https://drive.google.com/drive/folders/1nTq44CzOqEVAwUhtnmbdOUf4FRFBaBCZ?usp=sharing

a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

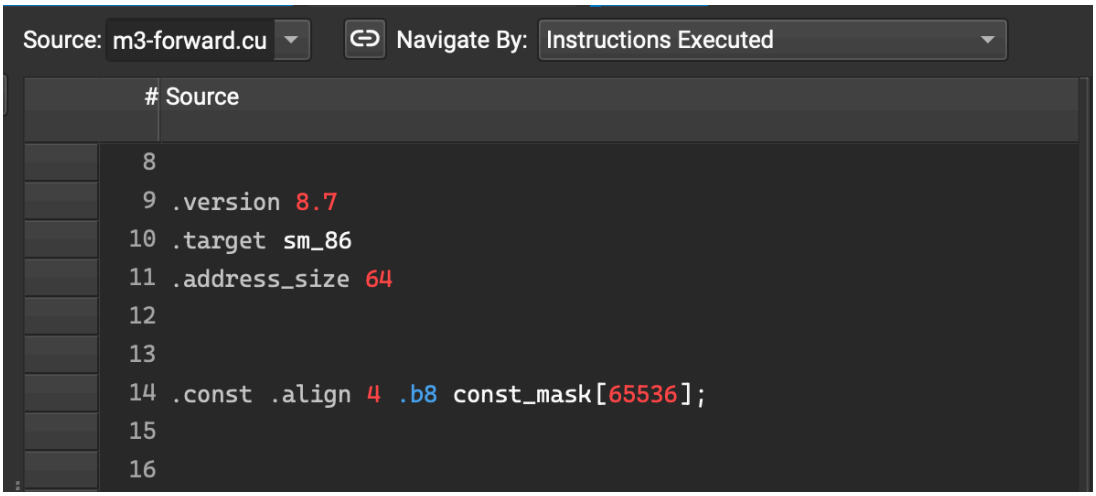
Putting the weight matrix in constant memory optimizes convolution because the kernel weights are never modified and reused by many threads. Instead of every thread repeatedly loading the same filter values from global memory, the weights can be cached in constant memory and broadcast efficiently when threads read the same address. This should reduce global memory traffic and improve performance, especially when many output pixels reuse the same K × K mask values. The expected behavior is lower memory-load overhead for the mask, faster convolution time, and unchanged accuracy because the same weights are used, just stored in a faster read-only memory space.

b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.

Code Snippet:

```
#define MAX_CONST_MASK_FLOATS 16384  // 64KB
__constant__ float const_mask[MAX_CONST_MASK_FLOATS];
```

NCU PTX Source:



I verified that the mask was placed in constant memory by checking the PTX source in Nsight Compute. The PTX contains `.const .align 4 .b8 const_mask[65536];`, which shows that const_mask was allocated in the constant memory address space with a 64 KB allocation. This matches the CUDA declaration `__constant__ float const_mask[16384]`.

NCU Memory Workload Analysis(compared to baseline):

The Memory Workload Analysis suggests that constant memory reduced pressure on the memory subsystem. Although the L1/TEX and L2 hit rates changed very little, the optimized version showed a large decrease in memory busy percentage, max bandwidth usage, and memory pipe busy percentage. This indicates that the kernel was spending less of its active time limited by memory-system activity, which is consistent with moving the small, repeatedly reused mask from global memory into constant memory. The improvement is not dramatic which leads me to think it won't help op times much.

c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	*0.59289 ms*	*0.508562 ms*	*1.101452 ms*	*0.86*
1000	*5.69604 ms*	*4.93897 ms*	*10.63501 ms*	*0.886*
10000	*56.0601 ms*	*48.7368 ms*	*104.7969 ms*	*0.8714*

Not really. I expected constant memory to help because the convolution weights are small, read-only, and reused across many output pixels, but the measured performance is mixed. For batch size 100, total execution time improved from 1.29293 ms to 1.101452 ms, but for batch sizes 1000 and 10000, total time got worse: 8.97519 ms -> 10.63501 ms and 87.9945 ms -> 104.7969 ms.

The profiling explains why the speedup was limited. NCU confirmed that the mask was placed in constant memory, and the memory workload results showed much lower memory busy / memory pipe busy percentages, which suggests the optimization reduced some pressure on the memory system. However, the L1/TEX and L2 hit rates barely changed, and the kernel still performs many input loads, shared-memory accesses, synchronizations, and implicit-index calculations. That means mask loads were not the dominant bottleneck, especially for larger batches.

d. Does this optimization synergize with any other optimizations? How?

Constant memory can synergize with parameter sweeping because different tile shapes may change how often the same mask values are reused. It does not strongly synergize with streams, since streams mainly optimize host-device transfer overlap, while constant

memory optimizes device-side access. Constant memory made op-times worse so I'd say it doesn't really synergize well with any optimizations overall.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)

Already learned __constant__ in lecture. I didn't need any resources.

4. **Op_1: __restrict__ keyword**
https://drive.google.com/drive/folders/1gAH9AyYnVOAuKRAZJFOMFDobUox8arAW?usp=sharing

a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

The __restrict__ keyword tells the compiler that the main convolution pointers point to strictly separate memory regions. For example, marking input, mask, and output as __restrict__ means the compiler can assume that writing to output will not modify values in input or mask. This allows the compiler to perform more internal optimizations, such as avoiding unnecessary reloads from memory, improving instruction scheduling, and potentially using cached/read-only memory paths more effectively.

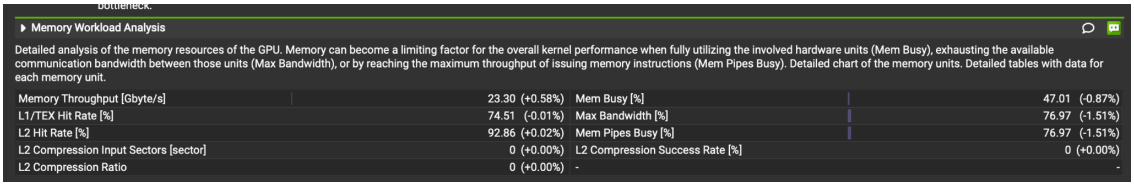
b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.

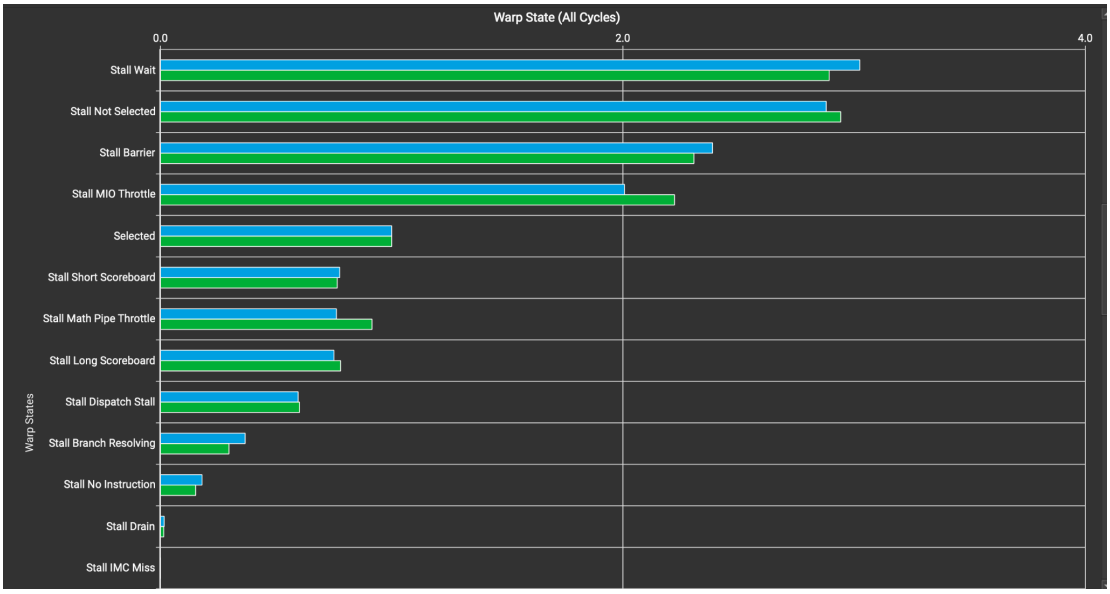
Code Snippet:

```
__global__ void matmul_conv_fused(const float *__restrict__ mask,
const float *__restrict__ input, float *__restrict__ output, int
Batch, int Map_out, int Channel, int Height, int Width, int K)
```

The `mask`, `input`, and `output` arrays point to separate GPU allocations, so it is safe to tell the compiler they do not alias.

NCU Warp State and Memory Workload Analysis:





c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	*1.42015 ms*	*0.3608 ms*	*1.78095 ms*	*0.86*
1000	*5.52758 ms*	*3.45085 ms*	*8.97843 ms*	*0.886*
10000	*54.7239 ms*	*33.8104 ms*	*88.5343 ms*	*0.8714*

The profiling results show that __restrict__ did not significantly improve the kernel. This makes sense because the compiler likely already had enough information, or the bottleneck was not pointer aliasing. The

optimization is still correct because the arrays are separately allocated and non-overlapping, but the measured benefit is essentially none

This is supported because NCU did not show a major change in instruction count or memory behavior, which is expected because __restrict__ is a compiler hint rather than an algorithmic change.

d. Does this optimization synergize with any other optimizations? How?

__restrict__ synergizes broadly with almost every kernel-level optimization because it gives the compiler more freedom to optimize memory operations. It's a no-brainer to use with optimizations like loop unrolling, though you may not see huge improvements in op-times because it's more of a compiler hint than an actual difference in convolution arithmetic.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)

https://developer.nvidia.com/blog/cuda-pro-tip-optimize-pointer-aliasing/

5. **Op_2: Loop unrolling**
https://drive.google.com/drive/folders/1h3oFEHskKCAIPosEknkmiZBNYXJ_QTN5?usp=sharing

a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

Loop unrolling optimizes the convolution kernel by reducing the overhead of loop control inside the inner accumulation loop. Instead of repeatedly checking the loop condition, incrementing the loop counter, and branching for every multiply-add, the compiler expands multiple iterations into straight-line code. This can improve instruction scheduling, expose more independent operations to the compiler, and reduce branch overhead in the tiled matrix multiplication step.

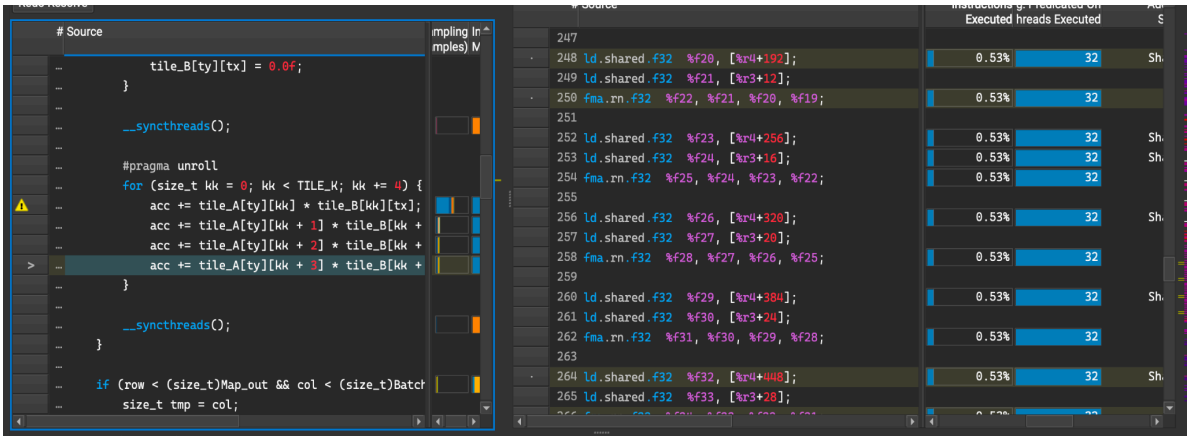
The expected behavior is probably a very limited speedup in op times, especially when the loop trip count is fixed and small.

b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.

Code Snippet:

```
#pragma unroll
for (size_t kk = 0; kk < TILE_K; kk += 4) {
      acc += tile_A[ty][kk] * tile_B[kk][tx];
      acc += tile_A[ty][kk + 1] * tile_B[kk + 1][tx];
      acc += tile_A[ty][kk + 2] * tile_B[kk + 2][tx];
      acc += tile_A[ty][kk + 3] * tile_B[kk + 3][tx];
}
```

NCU PTX Source:



Comparing the PTX showed that the inner accumulation loop was compiled into a straight-line sequence of shared-memory loads and `fma.rn.f32` instructions. This verifies that the `kk` loop was unrolled at the generated-code level: instead of a branch-controlled loop for each multiply-add, the compiler emitted all 16 multiply-add operations explicitly.

c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	*1.17826 ms*	*0.353271 ms*	*1.531531*	*0.86*
1000	*6.28415 ms*	*3.38968 ms*	*9.67383*	*0.886*
10000	*55.9653 ms*	*33.1567 ms*	*89.122 ms*	*0.8714*

Yes, the performance mostly matched my expectation after looking at the profiling results. Loop unrolling was expected to give only a small improvement because it does not change the amount of convolution work or memory traffic, it only reduces loop-control overhead in the inner accumulation loop.

In practice, the op times were basically the same because the compiler had already optimized the fixed `TILE_K = 16` loop. The PTX showed that both the baseline and manually unrolled versions compiled into a straight-line sequence of shared-memory loads and `fma.rn.f32` instructions, rather than a branch-controlled loop. That means my manual unrolling did not significantly change the generated machine-level computation.

d. Does this optimization synergize with any other optimizations? How?

Loop unrolling synergizes with every kernel level optimization, similar to __restrict__. It can also synergize with __restrict__ and parameter sweeping because all three help the compiler generate better straight-line code and schedule arithmetic more effectively. However, the benefit may be small if the compiler

already unrolls the loop automatically(as we saw) or if memory traffic and synchronization dominate runtime.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)

https://www.nvidia.com/docs/IO/116711/sc11-unrolling-parallel-loops.pdf

6. **Op_3: Sweeping various parameters to find best values (block sizes, amount of thread coarsening)**
https://drive.google.com/drive/folders/12ODUg2-K06J2O8X7kqUQDI69MOeLo_2s?usp=sharing

a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

Sweeping parameters optimize the convolution kernel by tuning values like block size, tile size, and thread coarsening to better match the GPU's hardware. Different choices change occupancy, register pressure, shared-memory usage, and how much work each thread performs. For example, larger tiles may improve data reuse but can reduce occupancy, while more thread coarsening can reduce similar types of loads but may increase register pressure. The expected behavior is that some configurations will perform noticeably better than others, and the best configuration is usually found experimentally.

b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.

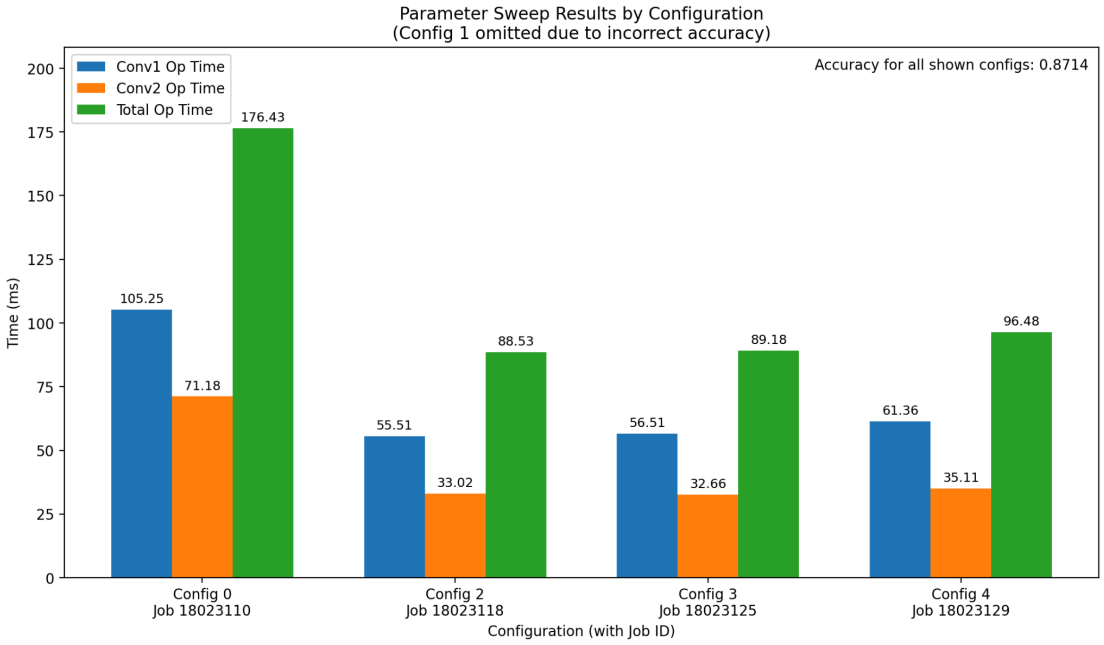
Code Snippet:

```
#ifndef SWEEP_CONFIG
#define SWEEP_CONFIG 3
#endif

#if SWEEP_CONFIG == 0
```

```
#define TILE_M 16
#define TILE_N 8
#define TILE_K 8
#define COARSEN_X 1
#elif SWEEP_CONFIG == 1
#define TILE_M 16
#define TILE_N 16
#define TILE_K 8
#define COARSEN_X 1
#elif SWEEP_CONFIG == 2
#define TILE_M 16
#define TILE_N 16
#define TILE_K 16
#define COARSEN_X 1
#elif SWEEP_CONFIG == 3
#define TILE_M 16
#define TILE_N 16
#define TILE_K 16
#define COARSEN_X 2
#elif SWEEP_CONFIG == 4
#define TILE_M 16
#define TILE_N 16
#define TILE_K 16
#define COARSEN_X 4
#endif
```

I implemented the parameter sweep by making the tile sizes and coarsening factor compile-time constants selected through SWEEP_CONFIG. Each configuration changes values such as TILE_M, TILE_N, TILE_K, and COARSEN_X, then recompiles the same fused implicit-GEMM convolution kernel with those parameters. This lets the same code test different block shapes and amounts of thread coarsening without rewriting the algorithm.

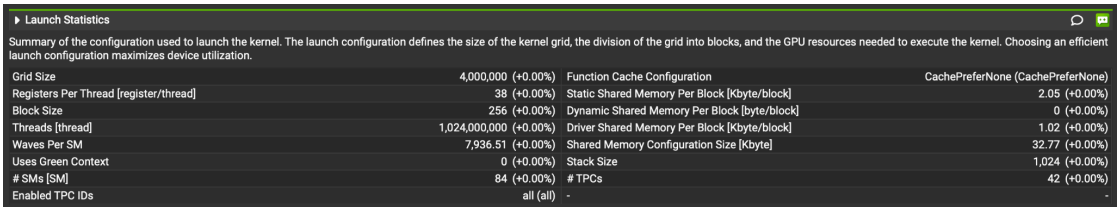
After profiling the best configuration we see that it almost exactly matches the baseline because the best configuration was the baseline.



Config Number	Conv1 Op Time	Conv2 Op Time	Total Op Time
0	105.253 ms	71.1805 ms	176.4335 ms
2	55.5143 ms	33.017 ms	88.5313 ms
3	56.5141 ms	32.6623 ms	89.1764 ms
4	61.3609 ms	35.1149 ms	96.4758 ms

Config 1 not included due to incorrect accuracy.

NCU Launch Statistics for Config 2(Compared to baseline):



Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	4,000,000 (+0.00%)	Function Cache Configuration	CachePreferNone (CachePreferNone)
Registers Per Thread [register/thread]	38 (+0.00%)	Static Shared Memory Per Block [Kbyte/block]	2.05 (+0.00%)
Block Size	256 (+0.00%)	Dynamic Shared Memory Per Block [byte/block]	0 (+0.00%)
Threads [thread]	1,024,000,000 (+0.00%)	Driver Shared Memory Per Block [Kbyte/block]	1.02 (+0.00%)
Waves Per SM	7,936.51 (+0.00%)	Shared Memory Configuration Size [Kbyte]	32.77 (+0.00%)
Uses Green Context	0 (+0.00%)	Stack Size	1,024 (+0.00%)
# SMs [SM]	84 (+0.00%)	# TPCs	42 (+0.00%)
Enabled TPC IDs	all (all)	-	-

c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	*1.3691 ms*	*0.35141 ms*	*1.72051 ms*	*0.86*
1000	*5.67811 ms*	*3.31685 ms*	*8.99496 ms*	*0.886*
10000	*57.3435 ms*	*32.5983 ms*	*89.9418*	*0.8714*

Yes, the performance mostly matched my expectations after the sweep. I expected that changing tile sizes and thread coarsening could improve performance, but the results showed that the original baseline configuration was already close to optimal for this kernel and GPU. Config 2 had the best total time at 88.5313 ms, while Config 3 was almost identical at 89.1764 ms, and Config 4 became slower at 96.4758 ms because additional coarsening likely increased register/shared-memory pressure without enough benefit.

The graph also shows that Config 0 was much worse, with a total time of 176.4335 ms, so the sweep was still useful: it confirmed that smaller tile settings like TILE_N = 8 and TILE_K = 8 were not a good fit. This was beneficial in showing that our baseline configuration was a good setting to continue with, and eliminating other configurations.

d. Does this optimization synergize with any other optimizations? How?

Parameter sweeping synergizes with almost everything because it finds the best tile sizes and coarsening factors for the current combination of optimizations. For example, Tensor Cores need tile sizes compatible with WMMA. It is also useful for identifying when an optimization hurts, my sweep showed that more coarsening or different tile sizes can increase overhead instead of improving

reuse. Sweeping through different parameters is a low cost, low implementation way of improving op-times or finding out what doesn't work in terms of configurations.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)

No reading needed. Fairly simple implementation.

7. **Op_5: Fixed point (FP16) arithmetic implementation**
https://drive.google.com/drive/folders/12rYbElg7pV5q37DzjmWLXD8HQRSgFgtF?usp=sharing

a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

FP16 optimizes the convolution kernel by reducing the cost of the multiply operations and lowering the amount of data needed to represent intermediate tile values. Instead of multiplying two FP32 values at a time, __half2 packs two FP16 values into one 32-bit register, allowing the kernel to process two multiply operations together with vectorized half-precision instructions. In the tiled implicit-GEMM convolution, this is useful because the inner accumulation loop repeatedly multiplies values from tile_A and tile_B, so packing pairs of values can reduce overhead and improve throughput.

The expected behavior is a speedup if the kernel is limited by arithmetic throughput. Accuracy may drop slightly because FP16 has less precision than FP32, but I've seen that accumulating the results back into a FP32 accumulator can reduce or nullify this error.

b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.

Code Snippets:

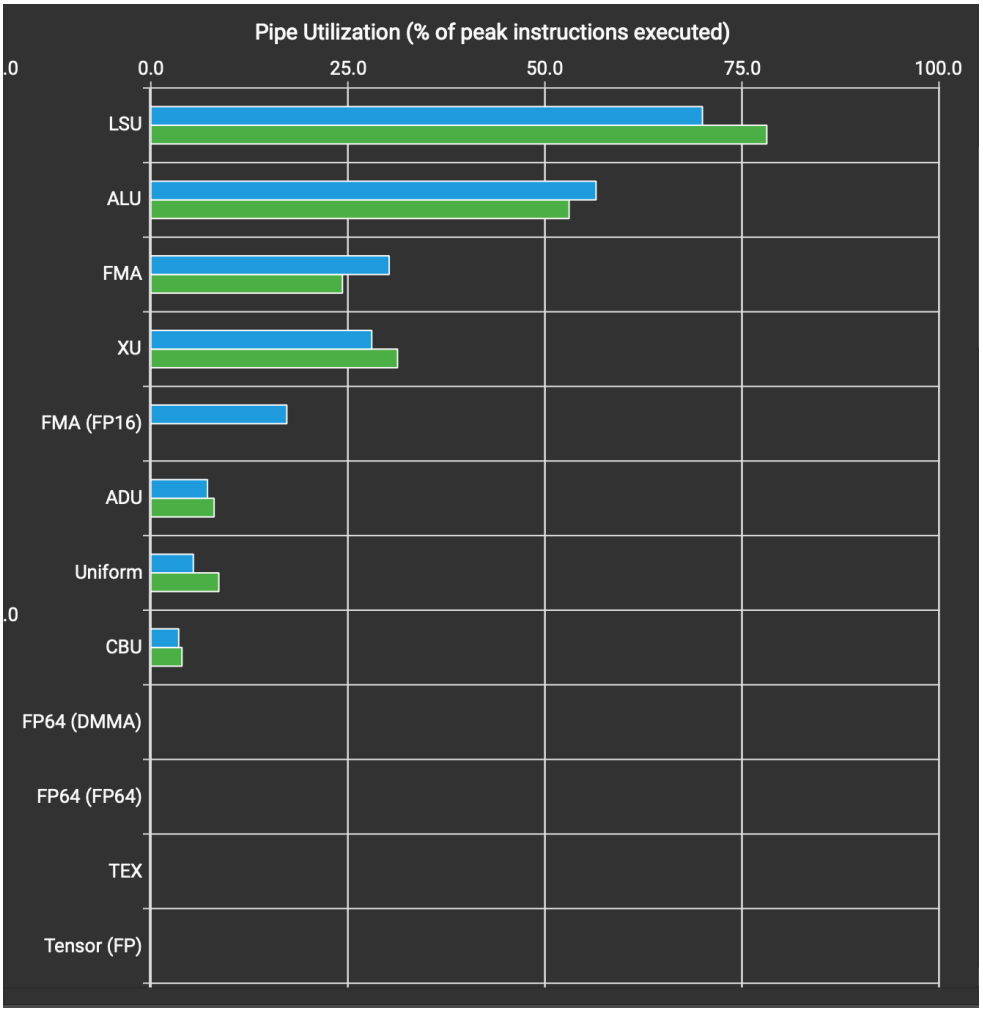
Shared Tile:

```
__shared__ __half tile_A[TILE_M][TILE_K];
__shared__ __half tile_B[TILE_K][TILE_N];
```

Multiply Loop:

```
for (size_t kk = 0; kk < TILE_K; kk += 2) {
    __half2 a2 = __halves2half2(tile_A[ty][kk], tile_A[ty][kk +
1]);
    __half2 b2 = __halves2half2(tile_B[kk][tx], tile_B[kk +
1][tx]);
    __half2 p2 = __hmul2(a2, b2);
    float2 pf = __half22float2(p2);
    acc += pf.x + pf.y;
}
```

I implemented the FP16 optimization by changing the shared-memory tiles from float to __half. The original input and mask arrays remain stored as FP32 in global memory, but when each tile is loaded into shared memory, the values are converted to FP16 using __float2half. In the inner accumulation loop, I process two products at a time by packing adjacent FP16 values into __half2 and multiplying them with __hmul2. The two FP16 products are then converted back to float and accumulated into a FP32 accumulator, preserving better numerical stability while still using packed FP16 arithmetic.



In Nsight Compute's Pipe Utilization section, the FP16 version shows nonzero utilization of the FMA (FP16) pipeline, while the baseline primarily uses the regular FMA/ALU/LSU pipelines. This confirms that the __half2 implementation is using half-precision arithmetic rather than only FP32 operations.

c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy

100	*1.37389 ms*	*0.395582 ms*	*1.769472 ms*	*0.86*
1000	*6.25105 ms*	*3.78095 ms*	*10.032 ms*	*0.886*
10000	*82.5283 ms*	*49.5832 ms*	*132.1115 ms*	*0.8714*

No, the performance did not match the ideal expectation. I expected half2 to improve runtime by doing two FP16 multiplies at once, but the measured time got worse: for batch size 10,000, the baseline was about 55 ms + 33 ms = 88 ms, while the FP16/half2 version was 82.5283 ms + 49.5832 ms = 132.1115 ms. Accuracy stayed the same at 0.8714, so correctness was preserved, but the optimization did not improve speed.

The NCU profile explains why. The Pipe Utilization screenshot shows that the FP16 version does activate the FMA (FP16) pipeline, so the half2 arithmetic is being used, however, the LSU and ALU pipelines are still heavily utilized. The code also converts FP32 values to FP16 when loading the shared tiles and then converts the half2 product back to FP32 for accumulation, so the conversion overhead can outweigh the benefit of packed FP16 multiplication.

d. Does this optimization synergize with any other optimizations? How?

   As mentioned before FP16/half2 synergizes with Tensor Cores because both rely on half-precision arithmetic. However, it may not synergize well with kernels dominated by indexing, memory movement, and synchronization, because FP16 only speeds up the arithmetic portion. In my results, FP16 was active in profiling, but the conversion overhead and memory/indexing bottlenecks outweighed the arithmetic benefit.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)

https://developer.nvidia.com/blog/mixed-precision-programming-cuda-8/

https://ion-thruster.medium.com/an-introduction-to-writing-fp16-code-for-nvidias-gpus-da8ac000c17f

https://docs.nvidia.com/cuda/cuda-math-api/cuda_math_api/group__CUDA__MATH____HALF__MISC.html